

IN2060 — Obligatorisk oppgave 3

fridatbe

22. oktober 2023

Oppgave 1

Fibonacci: part1.s

Oppgave 2

Fibonacci: part2.s

- Når man kaller en funksjon i Assembler er det noen registre kalleren av funksjonen må passe på å lagre unna fordi funksjonen kan endre verdien i registerene. Andre registre skal ha samme verdi etter et kall som de hadde før kallet. Under ARM funksjonskallkonvensjonen, hvilke registre er det kalleren av funksjonen som har ansvaret for å ta vare på verdiene i, og hvilke registre er det funksjonen som har ansvaret for?
 - Kaller/caller (ofte main) har reservert register 4-11, og trenger derfor ikke lagre dem på stakken for å bruke dem. I tillegg bruker caller R14 til LR (link register) hvor returadressen til funksjonen ligger, og R13 til SP (stack pointer) som peker til et sted i stakken. Dersom Caller skal bruke register 0-3, 12 eller stakken under SP, så må de lagres (push) på stakken og fjernes (pop) etterpå for å ikke overskrive de verdiene som funksjonen (callee) bruker.
 - Funksjonen/Callee har reservert registre 0-3, 12, og stakken under SP (alle de registerne som caller må lagre på stakken). I tillegg har callee også reservert CPSR (Current Program Status Register) som holder flaggverdier og kontrollverdier. Slik som med caller bør callee lagre registre som ikke er reservert for en selv på stakken. Callee må altså passe på at registerene som er caller reservert vil holde på de samme verdiene som før callee ble kalt på.
- Hva skjer med input argumentet til fib funksjonen etter at fib returnerer? (Hint: Tenk på hva som ligger i registeret der input argumentet lå tidligere)
 - Før fib ble kalt la main-funksjonen (caller) inn 13 i R0 (når man skal finne fib(13)). Og etterhvert som loopen i fib-funksjonen (callee) itereres gjennom, vil verdien i R0 minkes med én per gang. Og loopen er satt til å brytes når R0 inneholder verdier under 1. Så R0 vil inneholde 0 når fib returnerer.
- Hvilken endring måtte du gjøre i main fra Del 1 slik at programmet avsluttes riktig, og hvorfor måtte dette gjøres?
 - For at programmet skal avsluttes riktig bør LR legges på stakken fra main, slik at når BX LR blir kalt i fib hoppes det tilbake til main-funksjonen og ikke avslutter helt. Og andre gang BX LR blir kalt (i main), vil programmet avsluttes riktig fordi LR er tatt av stakken igjen på linja over

Oppgave 3

- Undersøk den produserte Assembler koden og sammenlign med din egen kildekode fra Del 2, er det noen store forskjeller mellom det du lagde og det GCC produserer?
 - GCC bruker CBZ (compare and branch if equal to zero) som er et betinget hopp, bestemt av flaggverdier. I motsetning til meg som brukte CMP og BMI.
 - Verdiene 1 og 0 har jeg lagt i register 1 og 2, mens GCC la dem i 2 og 3. I tillegg oppdaterer jeg R1 som en sum av R1 selv og R2, mens GCC legger senere summen over i R3
- Endre -Os til -O2, hvordan påvirker dette den produserte koden? Prøv å endre til -O3, ser du noen forandring nå?
 - La merke til at 1 og 0 ble lagret i R1 og R3 denne gangen
 - Løkken var også annerledes, ved at det ble brukt BNE
 - med O3: stort sett mye likt, men koden etter main er ulik. O3 legger blant annet inn 233 direkte med movs
 - Generelt skal Os gi kortere og mer kompakt kode enn O3 og O2 (funnet ut på nettet). Og Os har færre linjer med kode som man kan se, 12 linjer forskjell (til O2).
- Hvilke argumenter er det for og i mot å bruke en kompilator sammenlignet med å skrive Assembler? Tenk spesielt på hvor mye jobb det ville være å oversette programsnittene i Del 1, 2 og 3 til en annen prosessorarkitektur.
 - **For kompilator:** er mer dynamisk ved at det kan tilpasses til ulike prosessorarkitekturer, slipper å oversette koden. I tillegg er det enklere med tanke på abstraksjonsnivå, og derfor lettere å forstå for mennesker
 - **For Assembler:** har mer kontroll på hvor ting skal lagres og kan selv optimalisere ved behov, men må endres dersom det skal kjøre på ulike arkitekturer

Oppgave 4

Bilde av utregningene fra skriveboka 1-4

Del 4

① 2.0 i IEEE754:

2.0 i binær: 10
 2.0 i tenksteply: $1,0 \cdot 2^1$
 Biased eksponent: $127 + 1 = 128(10)$
 $= 10000000(2)$

Sign: 0 (positiv)

Fraction: fjernes 1 for komma i tenksteply fremstilling, så det blir 0

Sign: 0
 Biased eksponent: 10000000
 Fraction: 00000000000000000000000000000000

② 3.0 i IEEE754:

3.0 i binær: 11
 3.0 i tenksteply: $1,1 \cdot 2^1$
 Biased eksponent: $127 + 1 = 128(10)$
 $= 10000000(2)$

Sign: 0
 Biased eksponent: 10000000
 Fraction: 00000000000000000000000000000000

③ 0.50390625 som IEEE 754:

- 11 i binær: 0,10000001
 - 11 i tenksteply: $1,0000001 \cdot 2^{-1}$
 Biased eksponent: $127 + (-1) = 126(10)$
 $= 01111110(2)$

Sign: 0
 Biased eksponent: 01111110
 Fraction: 00000001000000000000000000000000

2.0 + 0.50390625:

Sign	Eksponent	Fraction
0	10000000	00000000000000000000000000000000
0	11111110	00000001000000000000000000000000

Differanse mellom eksponenter:

$$\begin{array}{r} 10000000 \\ - 01111110 \\ \hline 01000010 \end{array} = \underbrace{10000010}_{2(10)}$$

shifter med 2:

0	10000000	01000000000000000000000000000000
0	10000000	00100000010000000000000000000000

Legger sammen fractions:

$$\begin{array}{r} 1.000000000000000000000000 \\ + 0.010000000100000000000000 \\ \hline = 1.010000000100000000000000 \end{array}$$

Endelig svar: 2.50390625:

Sign	Eksponent	Fraction
0	10000000	01000000010000000000000000000000

Oppgave 5

Flyttall: part5.s

Resultater: addisjoner.txt